CS5383 · Proyecto 2 · Parte 2

G5 · Límites

Pruebas E2E de la interfaz, un pipeline que las corre solo y trazabilidad de cada historia hasta su ejecución

Kalos · Gianpier

UTEC · 2026-2

Contexto

Qolqe, el módulo G5 y lo que ya teníamos

La app

Qolqe es una billetera digital ficticia que da el curso. La probamos sin tocar su código.

Nuestro módulo

G5: el límite diario de transferencias y las alertas de consumo. 8 historias, más el núcleo de la cuenta.

Parte 1

108 escenarios de API en Gherkin con Playwright, contrato con Schemathesis y 9 defectos reportados.

La Parte 2 agrega lo que faltaba: probar lo que ve el cliente en la pantalla y que todo corra solo en cada cambio.

Contexto

Lo que pide la Parte 2, y dónde está

5	Todos los criterios definidos como web, automatizados	Selenium + pytest-bdd · lámina 4
2	Patrón mantenible, sin esperas fijas; pasa 3 veces seguidas	Page Object · lámina 5
5	Pipeline en cada push o PR: API + web en verde, con reportes publicados (y Schemathesis)	GitHub Actions · láminas 6 a 9
4	Trazabilidad HU → casos → ejecución, con los resultados del pipeline	láminas 10 y 11
4	Exposición con demo en vivo: el run, el reporte y la matriz	lámina 13

Las historias del G5, probadas en la pantalla

HU	Escenario de interfaz	Caso
G5-01 · 02 · 04	Ana baja su límite diario y crea una alerta de recargas (flujo feliz)	CP-57
G5-02	Un aumento entra en vigencia 24 horas después · bajar el límite pausa la alerta que queda encima	CP-58 · 59
G5-03	Una transferencia que supera el límite diario es rechazada	CP-60
G5-04	Validación del umbral al crear una alerta (6 ejemplos) · “Todos mis gastos” no supera el límite	CP-64 · 65
G5-06 · 07	Pausar, reanudar y eliminar una alerta · editar su umbral	CP-62 · 63
G5-08	Ana recibe un SMS al alcanzar el umbral de su alerta	CP-61
DEF-01 · 02 · 03	Los tres defectos del G5, vistos como los vería un cliente: fallan a propósito	CP-67 a 69

15 casos en `features/ui/limites_ui.feature`, Gherkin en español, más 3 de defectos conocidos. De las que el informe marcó como web, G5-06 y G5-07 están completas; G5-05 se ve en pantalla (tipo, umbral y estado), pero el gasto de hoy por alerta se verifica por API.

Page Object, sin esperas fijas y estable

tests/e2e/pages/limites_page.py · test_limites_ui.py

```
class LimitesPage(BasePage):
  LIMITE_DIARIO = tid("limite-diario")
  NUEVO_LIMITE = tid("input-nuevo-limite")

  def esperar_carga(self):
    self.espera.until(lambda d: ...text)

# el paso solo dice la intención
@when("cambia su límite diario a S/ {monto}")
def cambia_limite(driver, monto):
  LimitesPage(driver).cambiar_limite(monto)
```

Mantenible

Una clase por pantalla. Los selectores (`data-testid`) viven en un solo lugar: si cambia un botón, se arregla una línea.

Sin esperas fijas

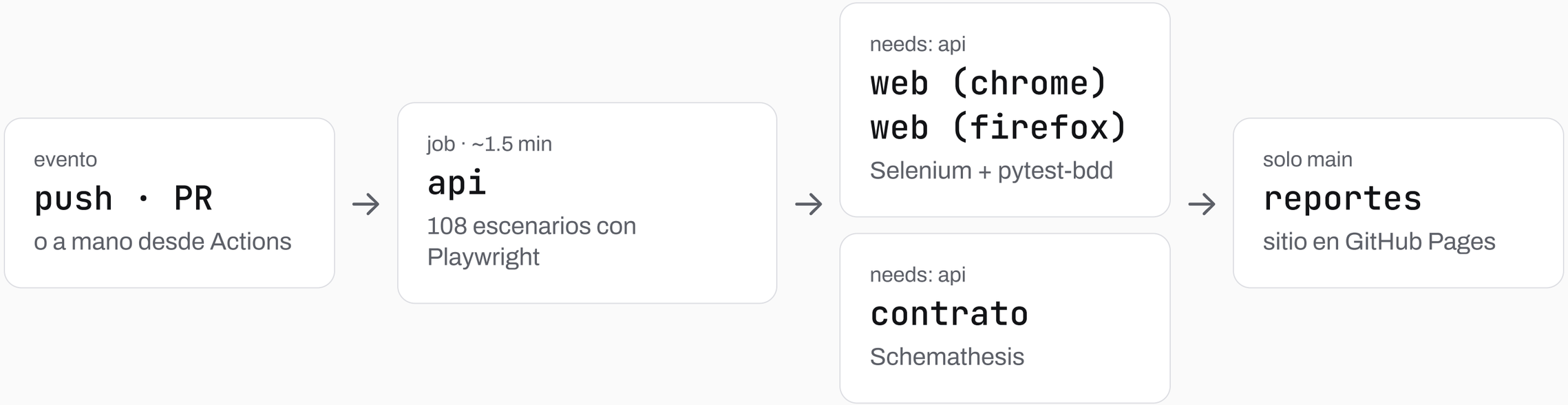
`WebDriverWait` espera a que la pantalla muestre el dato. No hay `sleep` en ningún paso.

Estable

Runs #21, #22 y #23 seguidas en verde, en Chrome y en Firefox.

2 · Pipeline

Lo rápido primero; cada etapa da más confianza



Si la API falla, no se gasta en la interfaz: web y contrato quedan como *skipped*. Archivo: `.github/workflows/pruebas.yml`.

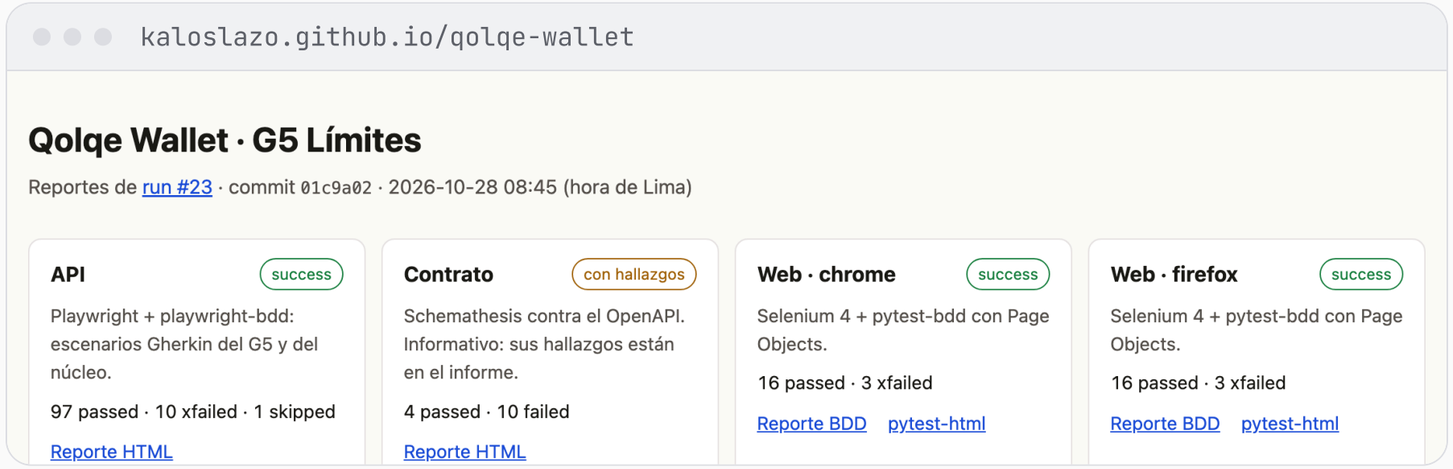
2 · Pipeline

Cada job deja evidencia que cualquiera puede abrir

Job	Qué corre	Qué deja	¿Bloquea?
`api`	Escenarios Gherkin de API	Reporte HTML, JUnit XML y JSON	Sí
`web` × 2	Escenarios de interfaz en Chrome y Firefox, sin ventana	Reporte BDD con captura por paso, pytest-html y JUnit	Sí
`contrato`	Schemathesis contra `/openapi.json`	Reporte HTML y JUnit	No: sus hallazgos ya están analizados
`reportes`	Junta todo y arma la portada y la matriz	Sitio en Pages y la matriz en el resumen	—

Además, cada job escribe una tabla en la página del run (`$GITHUB_STEP_SUMMARY`): se ve qué pasó sin descargar nada.

Los reportes, publicados en cada push a main



- Estado de cada job y sus cifras
- Reporte de Playwright, de Schemathesis y de Selenium en cada navegador
- La matriz de trazabilidad del mismo run
- Se publica también cuando algo falla: es cuando más hace falta

Quality gate: verde no significa “sin problemas”

Condición	Cómo se automatiza
Todas las pruebas de API y web pasan	El step termina con error si alguna falla
Los fallos conocidos están reportados	`@fail` en Playwright y `xfail` estricto en pytest, cada uno con su `@DEF-NN`. Si un defecto se arregla, la prueba también falla y avisa
No hay pruebas frágiles sin investigar	En CI, Playwright reintenta una vez; si al reintentar pasa, falla igual (`failOnFlakyTests`)
Nadie deja un `test.only` olvidado	`forbidOnly` en CI
Un PR en rojo no entra a main	Protección de rama con `api`, `web (chrome)` y `web (firefox)` obligatorios

3 · Trazabilidad

De la historia al resultado, sin llenar nada a mano

requisito
HU-G5-02
docs/requisitos.md

→

etiquetas en el .feature
@HU-G5-02 @CP-06
69 casos, CP-01 a CP-69

→

pipeline
JSON y JUnit XML
las etiquetas viajan con el resultado

→

matriz
run #23
Pages y docs/trazabilidad.md

- Playwright guarda las etiquetas de cada test en su reporte JSON
- En Selenium, el `conftest.py` las escribe como propiedad del JUnit XML
- `.github/scripts/trazabilidad.py` cruza los `.feature` con esos resultados: cada run produce su propio ciclo de ejecución, con fecha, commit y evidencia

3 · Trazabilidad

Cada caso, con su resultado del run

HU	Caso	Nivel	Resultado (run #23)
HU-G5-01	CP-01 Veo mis límites con lo usado y lo disponible del día	api	passed
HU-G5-02	CP-02 Validación del nuevo límite diario (valores límite)	api	passed 8/8
HU-G5-02	CP-06 Una reducción cancela el aumento pendiente	api	xfailed → DEF-01
HU-G5-02	CP-67 Una reducción del límite cancela el aumento programado	web	chrome · firefox: xfailed → DEF-01
HU-G5-04	CP-64 Validación del umbral al crear una alerta	web	chrome 6/6 · firefox 6/6
HU-N-07	CP-50 Validación del monto por operación (RN-05)	api	passed 5/5

Extracto. La matriz completa (núcleo y G5, 72 filas) está publicada en Pages y en `docs/trazabilidad.md`. Amarillo: falla por un defecto ya reportado.

Run #23: API y web en verde

97

passed en API

+ 10 defectos conocidos

16

passed en Chrome

+ 3 defectos conocidos

16

passed en Firefox

+ 3 defectos conocidos

10

operaciones con hallazgos de contrato

informativos

El 1 escenario omitido en API es de concurrencia y va a la Parte 3 (rendimiento). Tres runs seguidos en verde: #21, #22 y #23.

5 · Demo en vivo

Cuatro cosas, tres minutos

1 **Un run real del pipeline**

Actions › Pipeline › *Run workflow*. Se ven los jobs por etapas y el resumen de cada uno.

2 **El reporte abierto**

kaloslazo.github.io/qolqe-wallet → reporte BDD de Chrome: pasos, captura por paso y los 3 defectos en amarillo.

3 **La matriz de trazabilidad**

En la misma portada: cobertura por historia y el resultado de cada caso en ese run.

4 **La automatización, en vivo**

```
scripts/selenium.sh demo
```

Chrome hace el flujo feliz solo, resaltando cada elemento.

6 · Cierre

La alarma sonó sin que cambiáramos nada

Qué pasó

Salió pytest 9.2 y rompió pytest-bdd: los 15 escenarios web cayeron en CI con nuestro código intacto. En local seguía todo verde.

Qué hicimos

Lo vimos en el primer push, fijamos `pytest < 9.2` con el motivo en un comentario y el pipeline volvió a verde.

- El pipeline es la referencia, no la máquina de nadie
- Fijar versiones también es parte de que una prueba sea repetible
- Siguiente paso, Parte 3: rendimiento, con la concurrencia que hoy queda omitida

¿Preguntas?

github.com/kaloslazo/qolqe-wallet
kaloslazo.github.io/qolqe-wallet

Kalos · Gianpier